

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчёт

Лабораторная работа 3

Выполнил:
Воробьев Игорь

Группа J3210

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Перенести разработанное в первой и второй лабораторных работах приложение Project Manager на Vue.js и превратить его в одностраничное веб-приложение. Требования к результату: подключённый роутер, работа с внешним API через axios, разумное деление интерфейса на компоненты и вынесение повторяющейся функциональности в composable.

В итоговой версии сохранена предметная область сервиса управления проектами и командной работы. Реализованы стартовая страница, вход и регистрация, список проектов, страница отдельного проекта и страница 404. Сохранены и перенесены основные функции: поиск и фильтрация проектов, создание проекта, вступление в проект, управление участниками и ролями, работа с файлами, комментарии, а также светлая и тёмная темы и SVG-иконки.

Ход работы

Этап 1 – Проект и структура

Клиентская часть проекта собрана на Vue 3 и Vite. Приложение запускается на порту 8080. Данные вынесены из компонентов в отдельный HTTP-сервис на базе json-server, который работает на порту 3001 и использует файл server/db.json. Таким образом, Vue-приложение отвечает за интерфейс, а данные пользователей, проектов, файлов и комментариев получает через API.

Исходный код разделён по назначению:

- api — единый экземпляр axios и модули запросов users, projects, files и comments;
- router — маршруты приложения и общая проверка доступа перед переходом;
- layouts — общая обёртка внутренних страниц с шапкой, содержимым и подвалом;
- views — страницы SPA: главная, вход, регистрация, список проектов, проект и 404;
- components — переиспользуемые элементы интерфейса;
- composables — повторно используемая реактивная логика авторизации, темы и фильтрации;
- assets — общие стили и отдельные файлы переменных светлой и тёмной темы.

```
src/  
  api/          http.js, users.js, projects.js, files.js, comments.js  
  router/       index.js  
  composables/  useAuth.js, useTheme.js, useProjectFilters.js  
  layouts/      AppLayout.vue  
  views/        HomeView, LoginView, RegisterView, ProjectsView,  
                ProjectDetailsView, NotFoundView  
  components/   AppHeader, AppFooter, ThemeToggle, AppIcon,  
                ProjectCard, ProjectFilters, ProjectCreateModal,  
                TeamList, ProjectFiles, ProjectComments, AppAlert  
  assets/       main.css, themes/light.css, themes/dark.css  
  App.vue, main.js
```

Рис. 1: Структура клиентской части приложения после миграции на Vue.js.

Этап 2 — маршруты и контроль доступа

Для навигации подключён `vue-router`. Компоненты страниц загружаются лениво через динамический `import`, поэтому код конкретной страницы подгружается при переходе к ней. В приложении используются следующие маршруты:

- `/` — стартовая страница Project Manager;
- `/login` — вход;
- `/register` — регистрация;
- `/projects` — список проектов;
- `/projects/:id` — страница выбранного проекта;
- `/:pathMatch(.*)*` — страница «не найдено».

У маршрутов используются поля `meta.requiresAuth` и `meta.guestOnly`. Список проектов и отдельный проект доступны только авторизованному пользователю. Стартовая страница, вход и регистрация предназначены для гостя: если пользователь уже вошёл, роутер переводит его к проектам.

Этап 3 — запросы через `axios`

В файле `src/api/http.js` создан один экземпляр `axios` с базовым адресом `http://localhost:3001`, таймаутом 5 секунд и заголовком `Content-Type: application/json`. Обработка ошибок также находится в одном месте: `response interceptor` извлекает сообщение сервера и преобразует ошибку в единый формат.

Запросы разделены по сущностям, поэтому страницы не знают конкретные URL и не работают с `fetch` напрямую:

- `usersApi` — поиск пользователя по email и регистрация нового пользователя;
- `projectsApi` — получение списка и отдельного проекта, создание и частичное обновление;
- `filesApi` — получение файлов проекта, создание записи о файле и удаление;
- `commentsApi` — получение комментариев проекта и отправка нового комментария.

Этап 4 — деление на компоненты

После миграции повторяющиеся и логически самостоятельные части интерфейса вынесены в Vue-компоненты. Страницы из каталога `views` собирают интерфейс из готовых блоков и передают им только нужные данные и обработчики.

- `AppLayout` — общий каркас внутренних страниц. Внутреннее содержимое передаётся через `slot`, а шапка и подвал описаны один раз;
- `AppHeader` и `AppFooter` — общие элементы навигации; в шапке размещены имя пользователя, выход и переключатель темы;
- `ThemeToggle` и `AppIcon` — переключение темы и централизованный вывод SVG-иконок;
- `ProjectCard` — карточка проекта. Получает `project` и роль пользователя через `props` и сообщает наружу события `open` или `join`;
- `ProjectFilters` — поиск и фильтры по статусу, приоритету и исполнителю. Значения связаны со страницей через `v-model`;

- `ProjectCreateModal` — форма создания проекта в модальном окне;
- `TeamList` — состав команды, добавление участника, изменение роли и удаление;
- `ProjectFiles` — список файлов и события добавления/удаления;
- `ProjectComments` — список комментариев и форма отправки;
- `AppAlert` — единый вывод ошибок.

Данные передаются дочернему компоненту через `props`. Сам компонент не меняет состояние родителя напрямую, а сообщает о действии через `emit`. Например, `TeamList` сообщает `invite/change-role/remove`. `AppLayout` использует `slot`, чтобы один каркас подходил разным страницам. В `ProjectFilters` применяется `v-model` с именованными параметрами `search`, `status`, `priority` и `assignee`.

Этап 5 — `composables`

Повторяющаяся реактивная логика вынесена в каталог `composables`. В проекте используются три `composable`-функции:

- `useAuth` — хранит текущего пользователя в общем `ref`, реализует вход, регистрацию и выход. Сессия синхронизируется с `localStorage`;
- `useTheme` — хранит выбранную тему, вычисляет признак `isDark`, учитывает системную тему при первом запуске и сохраняет выбор в `localStorage`;
- `useProjectFilters` — хранит значения поиска и фильтров, формирует список исполнителей и вычисляет `filteredProjects` через `computed`.

Этап 6 — перенос функциональности приложения

Основная функциональность предыдущих лабораторных работ сохранена, но теперь работает через реактивное состояние `Vue` и `API`-модули.

- **Авторизация.** Вход выполняется по `email` и паролю, регистрация создаёт пользователя через `API`, а информация о текущей сессии хранится в `localStorage`;
- **Проекты.** На странице проектов данные загружаются при монтировании компонента. Доступны поиск по названию и фильтры по статусу, приоритету и исполнителю;
- **Создание и вступление.** Новый проект создаётся через модальное окно, автор автоматически получает роль «Администратор». Пользователь также может вступить в чужой проект с ролью «Наблюдатель»;
- **Команда.** Администратор может добавить зарегистрированного пользователя по `email`, изменить роль между «Участник» и «Наблюдатель» или удалить участника;
- **Файлы.** На странице проекта можно добавлять записи о файлах и удалять их. Метаданные сохраняются в `mock API`;
- **Комментарии.** Комментарий отправляется в `API` вместе с автором и временем и сразу появляется в списке;
- **Темы.** Светлые и тёмные значения цветов описаны `CSS`-переменными в отдельных файлах. `useTheme` устанавливает атрибут `data-theme` на корневой элемент и запоминает выбор;

- SVG-иконки. Иконки солнца, луны и поиска выведены через единый AppIcon.vue. SVG-path хранится в компоненте, поэтому интерфейс не зависит от внешнего sprite-файла.

При загрузке страницы проекта данные самого проекта, файлов и комментариев запрашиваются параллельно через Promise.all. Роль текущего пользователя вычисляется через computed. В зависимости от роли Vue условно показывает административные элементы: форму приглашения, выбор роли и кнопку удаления участника.

Этап 7 — обзор страниц приложения

Стартовая страница

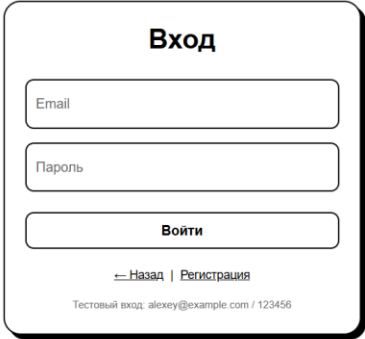
Маршрут /. Стартовый экран знакомит пользователя с приложением Project Manager. На странице расположены название сервиса, краткое описание, кнопки перехода ко входу и регистрации, а также переключатель светлой и тёмной темы. Логика работы: переходы выполняются через RouterLink без перезагрузки страницы; useTheme меняет тему и сохраняет выбор в localStorage. Если пользователь уже авторизован, router guard автоматически переводит его к списку проектов.



Рис. 2: Стартовая страница приложения.

Страница входа

Маршрут /login. Страница содержит форму авторизации по email и паролю. Логика работы: значения полей связаны с реактивным состоянием через v-model; при отправке проверяется заполнение полей и длина пароля. Затем useAuth.login находит пользователя через API, сохраняет текущую сессию в localStorage и выполняет переход на /projects. Ошибки выводятся через AppAlert.

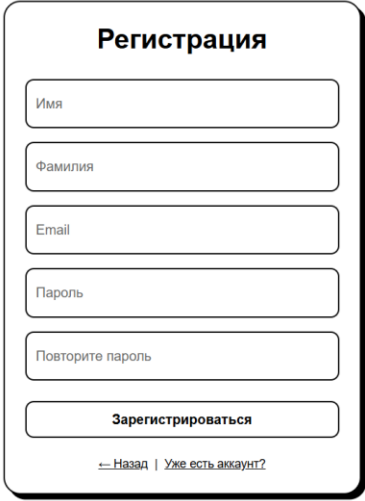


The image shows a login form titled "Вход" (Login) centered on a light gray background. The form is enclosed in a rounded rectangle with a thin black border. It contains two input fields: "Email" and "Пароль" (Password). Below the password field is a button labeled "Войти" (Login). At the bottom of the form, there are two links: "← Назад" (Back) and "Регистрация" (Registration). Below these links, there is a small text string: "Тестовый вход: alexey@example.com / 123456". In the top right corner of the page, there is a small gear icon.

Рис. 3: Форма входа в приложение.

Страница регистрации

Маршрут /register. Пользователь вводит имя, фамилию, email, пароль и подтверждение пароля. Логика работы: перед отправкой проверяются обязательные поля, длина и совпадение паролей. useAuth.register проверяет уникальность email через API, создаёт пользователя, сохраняет данные сессии в localStorage и перенаправляет пользователя к проектам.

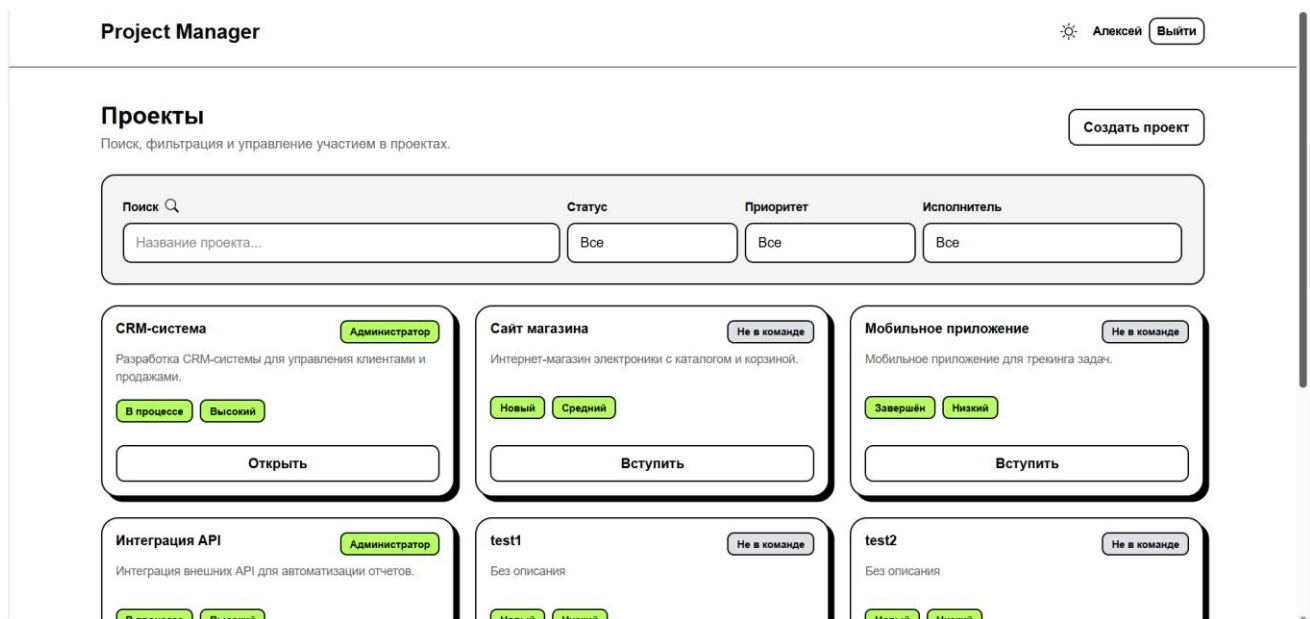


The registration form is titled "Регистрация". It contains five input fields: "Имя", "Фамилия", "Email", "Пароль", and "Повторите пароль". Below the fields is a button labeled "Зарегистрироваться". At the bottom, there are two links: "← Назад" and "Уже есть аккаунт?". A small gear icon is located in the top right corner of the form container.

Рис. 4: Форма регистрации нового пользователя.

Страница проектов

Маршрут /projects. Основная рабочая страница показывает карточки проектов. Логика работы: при открытии страницы проекты загружаются через projectsApi. Composable useProjectFilters реактивно фильтрует список по поиску, статусу, приоритету и исполнителю. Открытие карточки переводит на страницу проекта, вступление обновляет состав команды через API, а создание проекта отправляет новую запись на сервер и сразу добавляет её в список.



The screenshot shows the "Project Manager" interface. At the top, there's a header with "Project Manager" on the left and a user profile "Алексей" with a "Выйти" button on the right. Below the header, the main section is titled "Проекты" with a subtitle "Поиск, фильтрация и управление участием в проектах." and a "Создать проект" button. A filter bar contains a search input "Поиск" with a magnifying glass icon and a placeholder "Название проекта...", and three dropdown menus for "Статус" (set to "Все"), "Приоритет" (set to "Все"), and "Исполнитель" (set to "Все"). Below the filter bar, there are six project cards arranged in a 2x3 grid. Each card has a title, a description, a role indicator (e.g., "Администратор", "Не в команде"), status and priority tags (e.g., "В процессе", "Высокий", "Новый", "Средний", "Завершён", "Низкий"), and an action button (e.g., "Открыть", "Вступить").

Рис. 5: Страница со списком проектов и фильтрами.

Страница проекта

Маршрут `/projects/:id`. Страница отображает название, описание, статус, приоритет и дедлайн выбранного проекта. Логика работы: идентификатор берётся из параметра маршрута, после чего проект, файлы и комментарии параллельно загружаются через API. Роль текущего пользователя вычисляется по составу команды: администратор может приглашать участников, менять роли и удалять пользователей. Переключение вкладок меняет компонент файлов или комментариев, а добавление и удаление данных сразу синхронизируется с API и локальным состоянием.

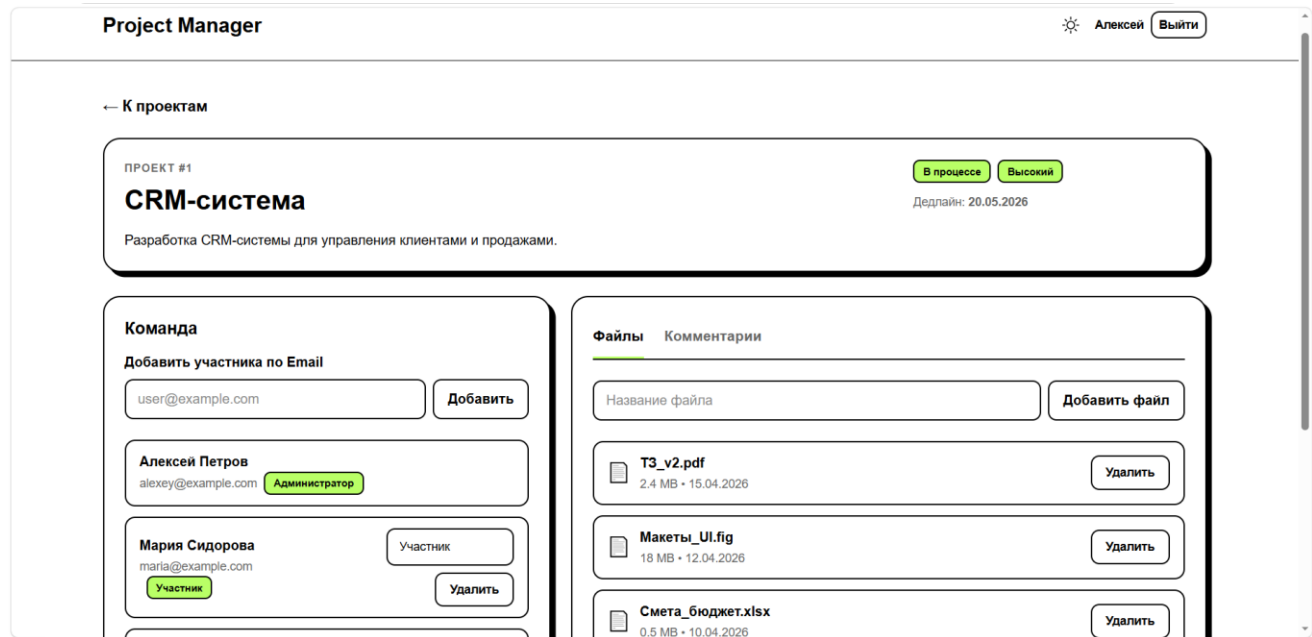


Рис. 6: Страница выбранного проекта.

Страница 404

Маршрут `/:pathMatch(.*)*`. Страница показывается при переходе по неизвестному адресу. Логика работы: catch-all маршрут Vue Router перехватывает любой URL, который не совпал с известными маршрутами, и отображает компонент 404. Кнопка «На главную» через RouterLink возвращает пользователя на стартовую страницу без полной перезагрузки приложения.

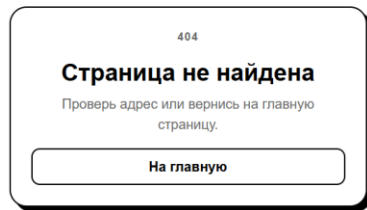


Рис. 7: Страница для неизвестного маршрута.

Вывод

Project Manager перенесён на Vue.js и преобразован в SPA. Навигацию выполняет Vue Router, запросы к mock API - axios; интерфейс разделён на компоненты, а повторяющаяся логика - на composables. Функции ЛР1-ЛР2 сохранены, а ручная работа с DOM заменена реактивным состоянием Vue.